What is an Array in Java?

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**. Each element is accessed using an **index**, which starts from **0**.

Simple Definition (Interview)

An array is a fixed-size collection of elements of the same data type stored in contiguous memory locations and accessed using an index.

Why Do We Use Arrays?

Without an array:

int marks1 = 80;

int marks2 = 75;

int marks3 = 90;

int marks4 = 85;

int marks5 = 95;

With an array:

int[] marks = {80, 75, 90, 85, 95};

Instead of creating many variables, you store all values in one array.

Array Syntax

Declaration

int[] numbers;

Creation

numbers = new int[5];

Declaration + Creation

int[] numbers = new int[5];

Initialize an Array

int[] numbers = {10, 20, 30, 40, 50};

Memory Representation

Index 0 1 2 3 4

+----+----+----+----+----+

Array --> | 10 | 20 | 30 | 40 | 50 |

+----+----+----+----+----+

Accessing Elements

```
int[] numbers = {10, 20, 30, 40, 50};


System.out.println(numbers[0]);
System.out.println(numbers[2]);
System.out.println(numbers[4]);
```

Output

10

30

50

Changing an Element

```
int[] numbers = {10, 20, 30};


numbers[1] = 100;


System.out.println(numbers[1]);
```

Output

100

Loop Through an Array

```
int[] numbers = {10, 20, 30, 40, 50};


for (int i = 0; i < numbers.length; i++) {
   System.out.println(numbers[i]);
}
```

Output

10

20

30

40

50

Enhanced For Loop

```
int[] numbers = {10, 20, 30, 40, 50};


for (int num : numbers) {
   System.out.println(num);
}
```

Types of Arrays

1. One-Dimensional Array

```
int[] numbers = {10, 20, 30};
```

Memory:

```
+----+----+----+
|10  |20  |30  |
+----+----+----+
```

---

2. Two-Dimensional Array

```
int[][] matrix = {
   {1, 2},
   {3, 4}
};
```

Memory:

1 2

3 4

Advantages

- Stores many values in one variable.

- Easy to access using indexes.
- Fast access to elements.
- Reduces code duplication.

Disadvantages

- Fixed size (cannot grow after creation).
- Stores only one data type.
- Inserting or deleting elements is not convenient.

Real-Life Example

Imagine a classroom with 5 students' marks:

Roll No. Marks

0 80

1 75

2 90

3 85

4 95

In Java:

```
int[] marks = {80, 75, 90, 85, 95};
```

Complete Example

```
public class ArrayExample {
   public static void main(String[] args) {

      int[] marks = {80, 75, 90, 85, 95};

      System.out.println("First Mark: " + marks[0]);
      System.out.println("Third Mark: " + marks[2]);

      System.out.println("\nAll Marks:");
```

```
        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }
    }
}
```

Output

First Mark: 80

Third Mark: 90

All Marks:

80

75

90

85

95

An array is a fixed-size collection of elements of the same data type stored in contiguous memory locations. It allows us to store multiple values in a single variable, and each element is accessed using an index that starts from 0.

Key Points to Remember

- ✅ Stores **same data type** only.
- ✅ Index starts from **0**.
- ✅ Fixed size.
- ✅ Stored in **contiguous memory**.
- ✅ Access elements using array[index].
- ✅ Get the array size using array.length.

In Java, arrays are mainly of **3 types**:

1. **One-Dimensional (1D) Array**

2. **Two-Dimensional (2D) Array**
3. **Jagged Array (Ragged Array)**

---

1. One-Dimensional (1D) Array

A **1D array** stores elements in a **single row**.

Syntax

```
dataType[] arrayName = new dataType[size];
```

Example

```
public class OneDArray {
   public static void main(String[] args) {

      int[] numbers = {10, 20, 30, 40, 50};

      for (int i = 0; i < numbers.length; i++) {
         System.out.println(numbers[i]);
      }
   }
}
```

Output

```
10
20
30
40
50
```

Memory Representation

```
Index     0    1    2    3    4
        +----+----+----+----+----+
Array --> |10  |20  |30  |40  |50  |
        +----+----+----+----+----+
```

---

2. Two-Dimensional (2D) Array

A **2D array** stores data in **rows and columns** (like a table or matrix).

Syntax

```
dataType[][] arrayName = new dataType[rows][columns];
```

Example

```
public class TwoDArray {
   public static void main(String[] args) {


      int[][] numbers = {
         {1, 2, 3},
         {4, 5, 6}
      };


      for (int i = 0; i < numbers.length; i++) {
         for (int j = 0; j < numbers[i].length; j++) {
            System.out.print(numbers[i][j] + " ");
         }
         System.out.println();
      }
   }
}
```

Output

```
1 2 3
4 5 6
```

Memory Representation

```
       Column
       0   1   2
     +---+---+---+
Row 0 | 1 | 2 | 3 |
     +---+---+---+
Row 1 | 4 | 5 | 6 |
     +---+---+---+
```

3. Jagged Array (Ragged Array)

A **Jagged Array** is a 2D array where **each row can have a different number of columns**.

Syntax

```
int[][] arr = new int[3][];
```

Example

```
public class JaggedArray {
   public static void main(String[] args) {

      int[][] arr = new int[3][];

      arr[0] = new int[]{10, 20};
      arr[1] = new int[]{30, 40, 50};
      arr[2] = new int[]{60};

      for (int i = 0; i < arr.length; i++) {
         for (int j = 0; j < arr[i].length; j++) {
            System.out.print(arr[i][j] + " ");
         }
         System.out.println();
      }
   }
}
```

Output

```
10 20
30 40 50
60
```

Memory Representation

Row 0 → 10 20

Row 1 → 30 40 50

Row 2 → 60

Notice that each row has a **different length**.

Difference Between 1D, 2D, and Jagged Arrays

Feature	1D Array	2D Array	Jagged Array
Structure	Single row	Rows and columns	Rows with different column sizes
Dimensions	1	2	2
Row Length	Not applicable	Same for all rows	Different for each row
Example	{10,20,30}	{{1,2},{3,4}}	{{1,2},{3,4,5},{6}}

Easy Way to Remember

1D Array

10 20 30 40 50

2D Array

1 2 3

4 5 6

Jagged Array

1 2

3 4 5

6

There are three common types of arrays in Java:

1. **One-Dimensional Array** – Stores elements in a single row.
2. **Two-Dimensional Array** – Stores elements in rows and columns with equal column sizes.
3. **Jagged Array (Ragged Array)** – A 2D array in which each row can have a different number of columns.

Here are simple Java programs for **Sum, Subtraction, Multiplication, Even Number, and Odd Number**.

---

1. Sum of Two Numbers

```
import java.util.Scanner;


public class Sum {
   public static void main(String[] args) {


      Scanner sc = new Scanner(System.in);


      System.out.print("Enter first number: ");
      int a = sc.nextInt();


      System.out.print("Enter second number: ");
      int b = sc.nextInt();


      int sum = a + b;


      System.out.println("Sum = " + sum);


      sc.close();
   }
}
```

---

2. Subtraction of Two Numbers

```
import java.util.Scanner;


public class Subtraction {
   public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        int sub = a - b;

        System.out.println("Subtraction = " + sub);

        sc.close();
    }
}
```

---

3. Multiplication of Two Numbers

```
import java.util.Scanner;

public class Multiplication {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        int mul = a * b;
```

```
        System.out.println("Multiplication = " + mul);

        sc.close();
    }
}
```

---

4. Check Even or Odd Number

```
import java.util.Scanner;

public class EvenOdd {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        if (num % 2 == 0) {
            System.out.println(num + " is Even");
        } else {
            System.out.println(num + " is Odd");
        }

        sc.close();
    }
}
```

Example

Input

8

Output

8 is Even

Input

7

Output

7 is Odd

---

5. Sum of Array Elements

```
public class ArraySum {
   public static void main(String[] args) {

      int[] arr = {10, 20, 30, 40, 50};

      int sum = 0;

      for (int i = 0; i < arr.length; i++) {
         sum = sum + arr[i];
      }

      System.out.println("Sum = " + sum);
   }
}
```

Output

Sum = 150

---

6. Multiplication of Array Elements

```
public class ArrayMultiplication {
   public static void main(String[] args) {

      int[] arr = {2, 3, 4};

      int product = 1;
```

```
        for (int i = 0; i < arr.length; i++) {
            product = product * arr[i];
        }

        System.out.println("Product = " + product);
    }
}
```

Output

```
Product = 24
```

---

7. Sum of Even Numbers in an Array

```
public class EvenSum {
    public static void main(String[] args) {

        int[] arr = {1, 2, 3, 4, 5, 6};

        int sum = 0;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0) {
                sum = sum + arr[i];
            }
        }

        System.out.println("Sum of Even Numbers = " + sum);
    }
}
```

Output

```
Sum of Even Numbers = 12
```

---

8. Sum of Odd Numbers in an Array

```
public class OddSum {
   public static void main(String[] args) {

      int[] arr = {1, 2, 3, 4, 5, 6};

      int sum = 0;

      for (int i = 0; i < arr.length; i++) {
         if (arr[i] % 2 != 0) {
            sum = sum + arr[i];
         }
      }

      System.out.println("Sum of Odd Numbers = " + sum);
   }
}
```

Output

Sum of Odd Numbers = 9

These are common beginner Java programs that help you practice:

- Arrays
- for loops
- if-else
- Arithmetic operators (+, -, *)
- Modulus operator (%) for checking even and odd numbers.

What is a String in Java?

A **String** is a sequence of characters enclosed in **double quotes (" ")**.

A String is used to store text such as names, addresses, messages, etc.

Examples

```
String name = "Vithesh";
```

String city = "Ujire";

String college = "SDM";

Syntax

String variableName = "Value";

Example:

String name = "Rahul";

Memory Representation

String name = "Rahul";

Stack Memory Heap Memory

name ——————▶ "Rahul"

The variable name stores a **reference** to the String object.

Example Program

```
public class StringExample {
   public static void main(String[] args) {

      String name = "Vithesh";

      System.out.println(name);
   }
}
```

Output

Vithesh

Taking String Input

```
import java.util.Scanner;
```

```
public class InputString {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter your name: ");
      String name = sc.nextLine();

      System.out.println("Hello " + name);

      sc.close();
   }
}
```

Output

```
Enter your name: Vithesh
Hello Vithesh
```

Common String Methods

1. length()

Returns the number of characters.

```
String s = "Java";

System.out.println(s.length());
```

Output

```
4
```

---

2. toUpperCase()

Converts to uppercase.

```
String s = "java";

System.out.println(s.toUpperCase());
```

Output

JAVA

---

3. toLowerCase()

Converts to lowercase.

String s = "JAVA";

System.out.println(s.toLowerCase());

Output

java

---

4. charAt()

Returns the character at a given index.

String s = "Java";

System.out.println(s.charAt(2));

Output

v

(Index starts from **0**.)

---

5. equals()

Compares two strings.

String s1 = "Java";

String s2 = "Java";

System.out.println(s1.equals(s2));

Output

true

---

6. equalsIgnoreCase()

Ignores uppercase/lowercase differences.

```
String s1 = "JAVA";
String s2 = "java";

System.out.println(s1.equalsIgnoreCase(s2));
```

Output

```
true
```

---

7. contains()

Checks whether a string contains another string.

```
String s = "Java Programming";

System.out.println(s.contains("Java"));
```

Output

```
true
```

---

8. substring()

Extracts part of a string.

```
String s = "Programming";

System.out.println(s.substring(3, 7));
```

Output

```
gram
```

---

9. replace()

Replaces characters or words.

```
String s = "Java";

System.out.println(s.replace("Java", "Python"));
```

Output

```
Python
```

---

10. concat()

Joins two strings.

String first = "Hello";

String second = " World";

System.out.println(first.concat(second));

Output

Hello World

Difference Between == and equals()

String s1 = "Java";

String s2 = "Java";

System.out.println(s1 == s2);

System.out.println(s1.equals(s2));

Output

true

true

However:

String s1 = new String("Java");

String s2 = new String("Java");

System.out.println(s1 == s2);

System.out.println(s1.equals(s2));

Output

false

true

- == compares **references (memory addresses)**.
- equals() compares the **actual text (content)**.

String vs StringBuilder

String	StringBuilder
Immutable (cannot be changed)	Mutable (can be changed)
Slower for many modifications	Faster for many modifications
Use for fixed text	Use when repeatedly appending or modifying text

---

A String in Java is a sequence of characters used to store text. It is an object of the String class, is immutable (cannot be changed after creation), and provides many built-in methods such as length(), charAt(), substring(), equals(), and toUpperCase().

Java Program to Check Vowel or Consonant

A **vowel** is one of these letters:

- **A, E, I, O, U** (uppercase or lowercase)

Any other alphabet letter is a **consonant**.

Program

```
import java.util.Scanner;


public class VowelCheck {
   public static void main(String[] args) {


      Scanner sc = new Scanner(System.in);


      System.out.print("Enter a character: ");
      char ch = sc.next().charAt(0);


      if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U' ||
         ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {


         System.out.println(ch + " is a Vowel");


      } else {
```

```
            System.out.println(ch + " is a Consonant");
        }

        sc.close();
    }
}
```

Output 1

Input

a

Output

a is a Vowel

Output 2

Input

k

Output

k is a Consonant

Better Program (Using toLowerCase())

This avoids checking both uppercase and lowercase separately.

```
import java.util.Scanner;

public class VowelCheck {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a character: ");
        char ch = Character.toLowerCase(sc.next().charAt(0));

        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
```

```
            System.out.println("Vowel");
        } else {
            System.out.println("Consonant");
        }

        sc.close();
    }
}
```

A vowel is one of the five English letters: A, E, I, O, and U. In Java, we can check whether a character is a vowel by comparing it with these letters using the logical OR (||) operator or by converting the input to lowercase first.

What is an Exception in Java?

An **exception** is an event that occurs during program execution and **interrupts the normal flow of the program**.

If an exception is not handled, the program terminates abnormally.

Example

```
public class ExceptionExample {
    public static void main(String[] args) {

        int a = 10;
        int b = 0;

        System.out.println(a / b);
    }
}
```

Output

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
```

The program crashes because dividing by zero is not allowed.

How to Handle Exceptions?

Java provides **5 keywords** for exception handling:

- try
- catch
- finally
- throw
- throws

The most common is **try-catch**.

Example

```
public class ExceptionDemo {
   public static void main(String[] args) {

      try {
         int a = 10;
         int b = 0;

         System.out.println(a / b);

      } catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero.");
      }

      System.out.println("Program continues...");
   }
}
```

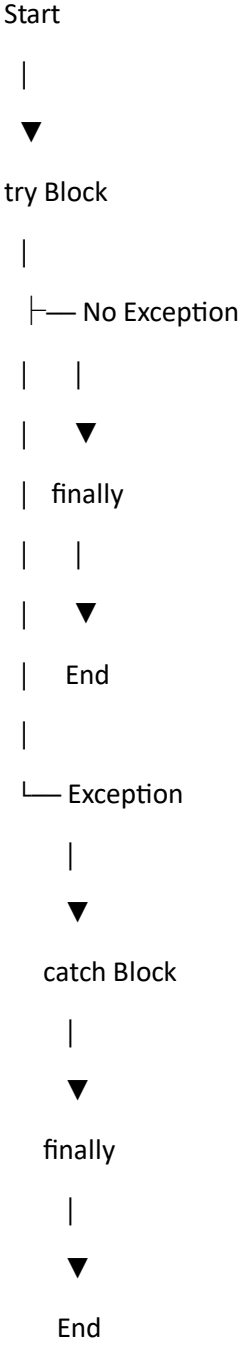
Output

Cannot divide by zero.

Program continues...

Without try-catch, the program would stop.

Exception Handling Flow



---

3 Types of Exceptions

There are **three main categories**.

1. Checked Exception

- Checked at **compile time**.
- Must be handled using try-catch or throws.

Examples:

- IOException
- SQLException
- FileNotFoundException

Example:

```
import java.io.FileReader;
import java.io.IOException;

public class CheckedExample {
   public static void main(String[] args) {

      try {
         FileReader file = new FileReader("test.txt");
      } catch (IOException e) {
         System.out.println("File not found.");
      }
   }
}
```

---

2. Unchecked Exception

- Occurs at **runtime**.
- The compiler does not force you to handle it.

Examples:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

Example:

```
int a = 10;
int b = 0;
```

System.out.println(a / b);

Output:

ArithmeticException

---

3. Errors

Errors are serious problems that applications usually **should not try to handle**.

Examples:

- OutOfMemoryError
- StackOverflowError

Example:

```
public class ErrorExample {

   static void display() {
      display();
   }

   public static void main(String[] args) {
      display();
   }
}
```

Output:

StackOverflowError

Difference Between the 3 Types

Type	Checked by	Must Handle?	Example
Checked Exception	Compiler	✅ Yes	IOException
Unchecked Exception	JVM (Runtime)	❌ No	ArithmeticException
Error	JVM	❌ No	OutOfMemoryError

Keywords Used in Exception Handling

Keyword	Purpose
try	Contains code that may cause an exception
catch	Handles the exception
finally	Executes whether an exception occurs or not
throw	Explicitly throws an exception
throws	Declares that a method may throw exceptions

Example Using finally

```
try {
   int a = 10 / 0;
} catch (ArithmeticException e) {
   System.out.println("Exception handled");
} finally {
   System.out.println("Finally block executed");
}
```

Output

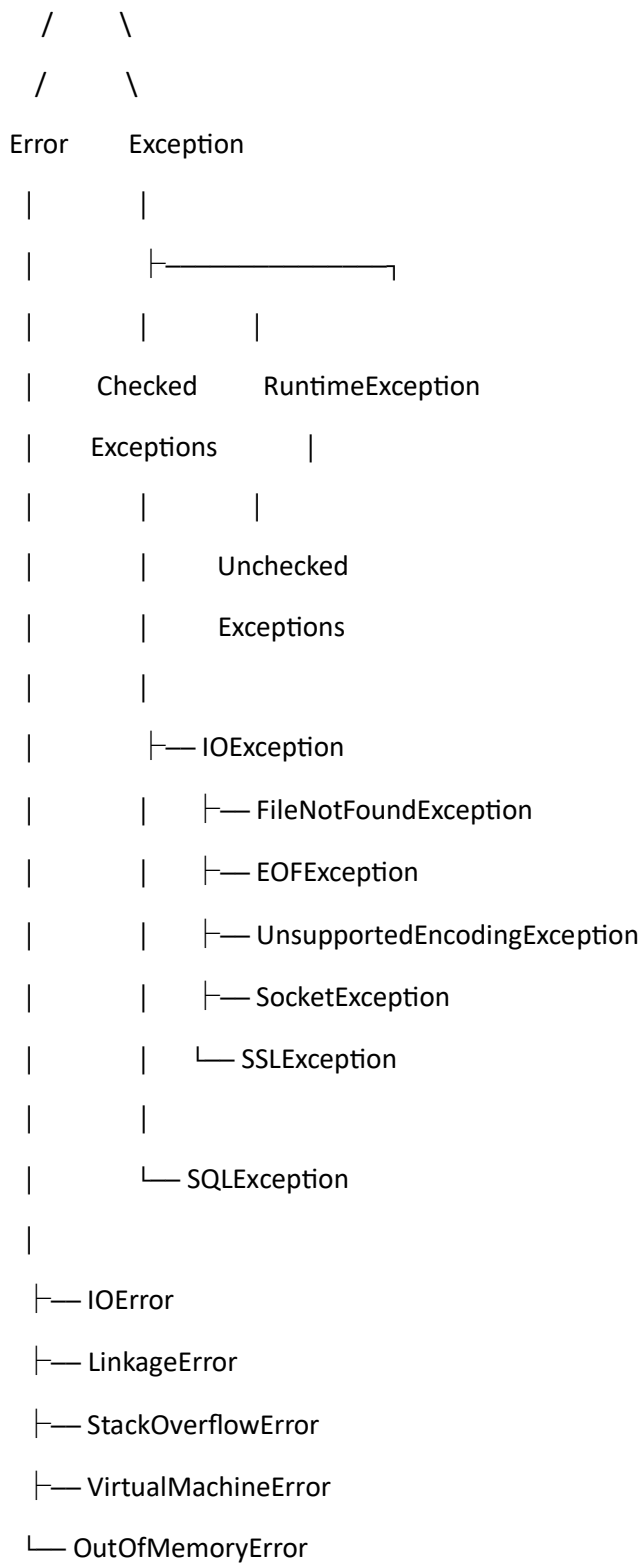
Exception handled

Finally block executed

The finally block runs even after an exception is caught.

An exception is an event that interrupts the normal execution of a Java program. Exceptions can be handled using try, catch, finally, throw, and throws. There are three main types: Checked Exceptions (handled at compile time), Unchecked Exceptions (occur at runtime), and Errors (serious JVM problems such as OutOfMemoryError and StackOverflowError).

Complete Java Exception Hierarchy



---

1. Object

class Object

- It is the **root (topmost) class** in Java.
- Every class in Java inherits from Object directly or indirectly.

Example:

String s = "Java";

Actually,

String

↑

Object

Similarly,

Student

↑

Object

Everything in Java is related to Object.

---

2. Throwable

class Throwable

Throwable is the parent class of everything that **can be thrown**.

Example

throw new ArithmeticException();

or

throw new IOException();

Both inherit from Throwable.

There are only **two children** of Throwable.

Throwable

Error Exception

---

3. Error

Errors are **serious problems**.

They are produced by

- JVM

- Operating System

Normally programmers **don't handle them**.

Example

Imagine your computer has only 100MB memory.

Your Java program asks for 500MB.

The JVM cannot provide it.

Then

OutOfMemoryError

occurs.

Common Errors

a) OutOfMemoryError

```
int arr[] = new int[999999999];
```

Output

OutOfMemoryError

Reason

No memory left.

---

b) StackOverflowError

```
public class Test{

static void demo(){
demo();
}

public static void main(String args[]){
demo();
}

}
```

Output

StackOverflowError

Reason

Method keeps calling itself forever.

---

c) LinkageError

Occurs when class loading fails.

Example

Wrong library versions.

---

d) VirtualMachineError

Occurs inside JVM.

Rare.

---

e) IOError

Occurs during I/O operations.

Rare.

Error Summary

Error	**Reason**
OutOfMemoryError	Heap memory full
StackOverflowError	Stack memory full
LinkageError	Class loading problem
VirtualMachineError	JVM failure
IOError	I/O failure

---

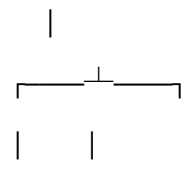
4. Exception

Exceptions are **problems that programmers can handle**.

Exception

has **2 categories**

Exception



---

5. Checked Exception

Checked by

Compiler

Compiler says

"Handle this exception or I won't compile."

Example

```
import java.io.*;

public class Demo{

public static void main(String args[]) throws IOException{

FileReader fr=new FileReader("abc.txt");

}

}
```

If file doesn't exist

Compiler forces

try-catch

or

throws

Common Checked Exceptions

IOException

means

Input Output problem.

Children

IOException

↓

FileNotFoundException

↓

EOFException

↓

SocketException

↓

SSLException

↓

UnsupportedEncodingException

FileNotFoundException

Example

FileReader fr=new FileReader("hello.txt");

If file missing

FileNotFoundException

EOFException

EOF

=

End Of File

Occurs while reading after file ends.

SocketException

Network connection problem.

SSLException

Secure HTTPS connection failure.

SQLException

Database problem.

Example

Connection con=DriverManager.getConnection(...);

Wrong password

↓

SQLException

Checked Exception Summary

Exception	Meaning
IOException	File/Input Output
FileNotFoundException	File missing
EOFException	End of file
SQLException	Database error
SocketException	Network problem
SSLException	HTTPS error

---

6. RuntimeException

Also called

Unchecked Exception

Compiler ignores them.

Program crashes only while running.

ArithmeticException

int a=10/0;

Output

ArithmeticException

NullPointerException

String s=null;

System.out.println(s.length());

Output

NullPointerException

NumberFormatException

Integer.parseInt("abc");

Output

NumberFormatException

IndexOutOfBoundsException

int arr[]={10,20};

System.out.println(arr[5]);

Output

ArrayIndexOutOfBoundsException

ClassCastException

Object obj="Java";

Integer x=(Integer)obj;

Output

ClassCastException

IllegalArgumentException

Passing illegal value.

Example

Thread.sleep(-5);

Output

IllegalArgumentException

SecurityException

Occurs when Java security manager denies access.

Rare.

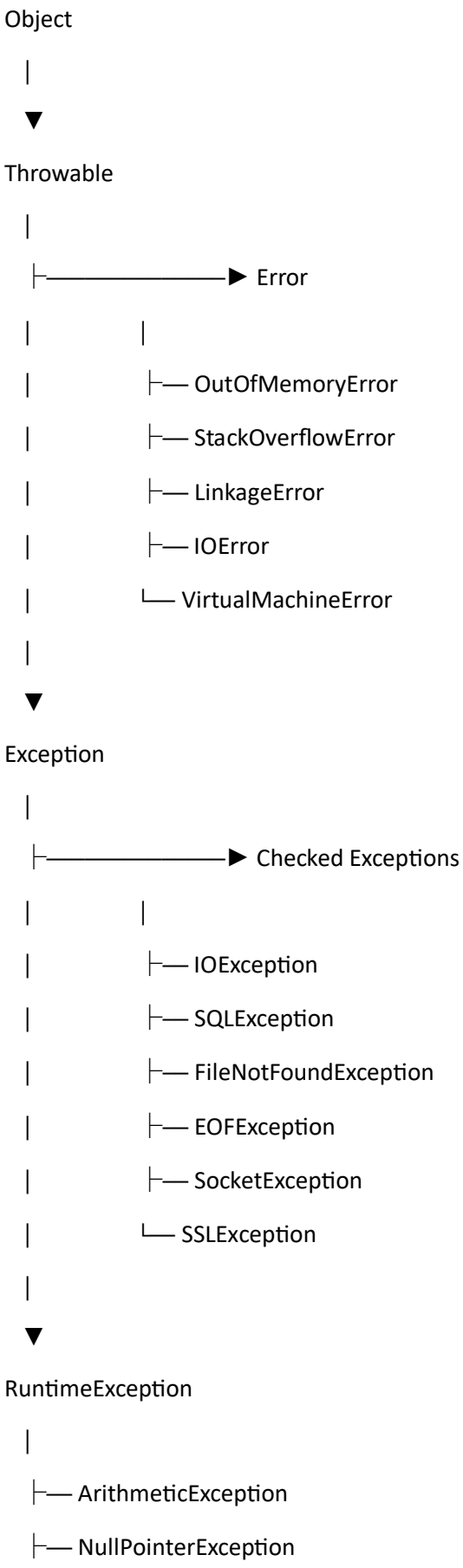
RuntimeException Summary

Exception	Reason
ArithmeticException	Divide by zero
NullPointerException	Using null object
NumberFormatException	Invalid number conversion
IndexOutOfBoundsException	Invalid index
ClassCastException	Wrong object casting
IllegalArgumentException	Invalid argument
SecurityException	Security violation

Difference Between Error, Checked Exception, and Unchecked Exception

Feature	Error	Checked Exception	Unchecked Exception
Checked by compiler	✗	✓	✗
Occurs	JVM	Compile time	Runtime
Must handle	✗	✓	✗
Programmer can usually fix?	Usually no	Yes	Yes
Example	OutOfMemoryError	IOException	NullPointerException

Complete Flow


Object
Throwable
Error
OutOfMemoryError
StackOverflowError
LinkageError
IOError
VirtualMachineError
Exception
Checked Exceptions
IOException
SQLException
FileNotFoundException
EOFException
SocketException
SSLException
RuntimeException
ArithmeticException
NullPointerException

├── NumberFormatException

├── ClassCastException

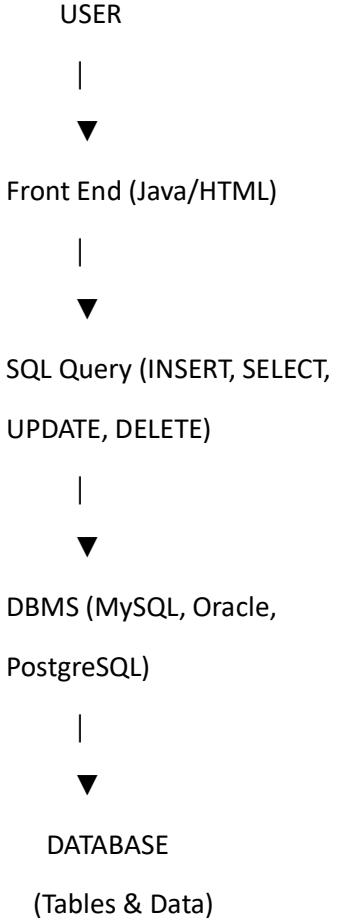
├── IllegalArgumentException

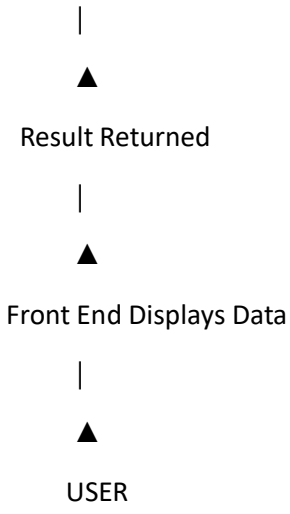
├── IndexOutOfBoundsException

└── SecurityException

In Java, every class ultimately inherits from Object. Throwable extends Object and is the base class for everything that can be thrown. Throwable has two main subclasses: Error and Exception. Error represents serious JVM problems such as OutOfMemoryError and StackOverflowError, which applications usually do not handle. Exception represents conditions that applications can handle. Exceptions are divided into **Checked Exceptions**, which are checked by the compiler and must be handled (for example, IOException and SQLException), and **Unchecked Exceptions**, which are subclasses of RuntimeException and occur at runtime (for example, ArithmeticException, NullPointerException, and NumberFormatException).

Basic DBMS Flow Chart




Result Returned
Front End Displays Data
USER